

CodeNergy Website ? Development Handover Report

#1) Project Overview

CodeNergy Website is a full-stack Next.js application that includes:

- A public marketing site with service pages, portfolio, blog, FAQ, contact, and careers pages.
- An admin panel for content and operations management (services, projects, users, team, testimonials, appointments, messages, FAQs, blog).
- Backend APIs built in the Next.js App Router ('src/app/api/') using MongoDB + Mongoose.
- Authentication based on JWT + cookie sessions for admin users.

#2) Tech Stack

##Frontend

- Next.js '^16.0.7'
- React / React DOM '^19.2.1'
- Tailwind CSS v4 + PostCSS
- Framer Motion, Swiper, React Icons, React Modal

##Backend / Data

- Next.js API route handlers ('route.js')
- MongoDB with Mongoose
- JWT auth ('jsonwebtoken', 'jose')
- Password hashing via 'bcryptjs'
- Email notifications via Nodemailer (SMTP)
- Cloudinary for image/media hosting

##State / Tooling

- Redux Toolkit + React Redux
- ESLint 9 + 'eslint-config-next'

#3) Repository Structure

- 'src/app/(site)': Public-facing site routes and layout.
- 'src/app/(admin)': Admin login + admin dashboard pages.
- 'src/app/api': Backend APIs (App Router route handlers).
- 'src/components/landings': Public site components.
- 'src/components/admin': Admin UI shell and editor components.
- 'src/models': Mongoose schemas/models.
- 'src/lib': shared utilities (DB, Cloudinary, auth helper, mailer).
- 'src/redux': Redux store and user slice.
- 'src/configs/middleware.js': middleware protecting admin dashboard routes.
- 'public/': static assets and uploaded media snapshots.

#4) Environment Variables (Required)

Create '.env.local' with at least:

- 'MONGODB_URI': primary MongoDB connection string.
- 'JWT_SECRET': signing secret for JWT token/cookie authentication.
- 'SMTP_HOST': SMTP host for outbound mail.
- 'SMTP_PORT': SMTP port.
- 'SMTP_USER': SMTP username.
- 'SMTP_PASS': SMTP password.
- 'CLOUDINARY_CLOUD': Cloudinary cloud name.
- 'CLOUDINARY_API_KEY': Cloudinary API key.
- 'CLOUDINARY_API_SECRET': Cloudinary API secret.

Note: a legacy/commented reference to 'MONGO_URI' exists in one blog route file, but active DB connection uses 'MONGODB_URI' from 'src/lib/mongodb.js'.

#5) NPM Scripts / Local Runbook

- Install: 'npm install'
- Development: 'npm run dev'
- Production build: 'npm run build'
- Start production server: 'npm run start'
- Lint: 'npm run lint'

#6) Runtime Architecture and Request Flow

1. Browser hits public pages or admin pages.
2. Public pages fetch content from '/api/*' routes.
3. Admin pages use cookie-based auth ('userInfo', 'token') and call protected APIs.
4. API handlers call 'connectDB()' and operate through Mongoose models.
5. Some mutating endpoints also:
 - write audit activity records to 'Activities' collection,
 - notify admins via SMTP email.
6. Uploaded images are stored through Cloudinary and/or local upload paths under 'public/uploads' depending on route logic.

#7) Authentication & Authorization

- Login endpoint validates email/password and sets cookies ('/api/auth/login').
- Middleware verifies JWT token cookie and blocks unauthenticated access for dashboard matcher paths.
- Server helper 'verifyAdmin()' checks 'userInfo' cookie and returns parsed user object.
- Most admin-write APIs call 'verifyAdmin()' and require role 'admin'; unauthorized requests return HTTP '401'.

#8) Public Site Routes

- '/' (home)
- '/aboutus'
- '/blog', '/blog/[slug]'
- '/careers'
- '/contactus'
- '/development', '/development/[id]'
- '/faq'
- '/marketing', '/marketing/[id]'
- '/ourphilosophy'
- '/outsourcing', '/outsourcing/[id]'
- '/portfolio', '/portfolio/[id]'
- '/pricing'
- '/team'
- '/technicalSupport', '/technicalSupport/[id]'

#9) Admin Routes

- '/login'
- '/admin/dashboard'
- '/admin/service'
- '/admin/projects'
- '/admin/team'
- '/admin/testimonials'

- '/admin/blog'
- '/admin/users'
- '/admin/messages'
- '/admin/appointments'
- '/admin/faqs'

#10) API Inventory (Methods + Purpose)

##Activities

- 'GET /api/Activities' ? list latest activities (admin-only).

##Appointments

- 'GET /api/appointments' ? list appointments (admin-only, populated service title).
- 'POST /api/appointments' ? create appointment (public), create activity, email admins.
- 'PATCH /api/appointments/[id]' ? update appointment (admin-only).
- 'DELETE /api/appointments/[id]' ? delete appointment (admin-only).

##Auth

- 'POST /api/auth/createUser' ? register user (public utility endpoint).
- 'POST /api/auth/login' ? login and set cookies.
- 'GET /api/auth/logout' ? clear auth cookies and logout.

##Blog

- 'POST /api/blog' ? create blog.
- 'GET /api/blog/[slug]' ? fetch single blog by slug.
- 'PUT /api/blog/[slug]' ? update blog by slug.
- 'DELETE /api/blog/[slug]' ? delete blog by slug.

##FAQ

- 'GET /api/faqs' ? list/search/paginate FAQs (public/admin depending query usage).
- 'POST /api/faqs' ? create FAQ (admin expected).
- 'PUT /api/faqs/[id]' ? update FAQ (admin expected).
- 'DELETE /api/faqs/[id]' ? delete FAQ (admin expected).

##Health

- 'GET /api/health' ? health check endpoint (currently also touches DB).

##Messages

- 'GET /api/messages' ? list messages (admin-only).
- 'POST /api/messages' ? create message from contact form, notify admins.

##Projects

- 'GET /api/projects' ? list projects.
- 'POST /api/projects' ? create project.
- 'GET /api/projects/[id]' ? get project detail.
- 'DELETE /api/projects/[id]' ? delete project.

##Services

- 'GET /api/services' ? list all services.
- 'POST /api/services' ? create service.
- 'GET /api/services/[id]' ? service by id.
- 'DELETE /api/services/[id]' ? delete service.
- 'GET /api/services/category/[category]' ? services by category.
- 'GET /api/services/forNavbar' ? grouped services response for navigation menu.

##Stats

- 'GET /api/stats' ? dashboard aggregate counters for entities.

##Team

- 'GET /api/team' ? list members.
- 'POST /api/team' ? create member.
- 'PUT /api/team/[id]' ? update member.
- 'DELETE /api/team/[id]' ? delete member.

##Testimonials

- 'GET /api/testimonials' ? list testimonials.
- 'POST /api/testimonials' ? create testimonial.
- 'PATCH /api/testimonials/[id]' ? update testimonial.
- 'DELETE /api/testimonials/[id]' ? delete testimonial.

##Users

- 'GET /api/users' ? list users (admin-only).
- 'POST /api/users' ? create user (admin-only).
- 'PUT /api/users/[id]' ? update user role (admin-only).
- 'DELETE /api/users/[id]' ? delete user (admin-only).

#11) Data Model Reference

##'User'

- 'name', 'email' (unique), 'password' (hashed), 'role' ('admin|user'), 'isActive', 'loginDetails[]', timestamps.

##'Services'

- 'image', 'imagePublicId', 'category', 'subcategory', 'title' (unique), 'href', 'description', 'sideDescription', 'longDescription', timestamps.

##'Projects'

- 'title', 'image', 'description', 'service' (ref Service).

##'Team'

- 'name', 'contact', 'image', 'designation', 'description', timestamps.

##'Testimonials'

- 'name', 'role', 'comment', 'rating (1-5)', timestamps.

##'Appointment'

- 'name', 'email', 'date', 'time', 'message', 'page', 'serviceId' (ref Service), 'isDone', timestamps.

##'Messages'

- 'name', 'email' (required), 'message', 'phone', 'seen', timestamps.

##'Blog'

- 'title', 'description', 'content', 'author' (ref User), 'image', 'slug' (unique), timestamps.

##'Faq'

- 'question', 'answer', 'visible', 'order', 'category', timestamps.

'Activities'

- 'content', 'link', timestamps.

#12) Frontend Notes

- Global public layout includes SEO metadata, navbar/footer, and WhatsApp floating button.
- Home page fetches services + testimonials from APIs and composes multiple landing sections.
- Service navigation is dynamic and multi-level in the navbar.
- Admin area is rendered through server layout + client provider for Redux hydration.
- Rich text/editor related admin components exist ('Editor.js', 'SlateDisplay.js') based on Slate.

#13) Operational Notes (Important for New Developer)

1. Auth cookies are central: keep cookie names/shape consistent ('token', 'userInfo').
2. Most admin APIs expect 'verifyAdmin()': maintain that contract for new endpoints.
3. DB connection helper is singleton-like ('isConnected') to avoid repeated connects.
4. Email notifications are triggered on several write operations; check SMTP in each environment.
5. Cloudinary domain is whitelisted in 'next.config.mjs' for image rendering.
6. Naming inconsistencies exist ('testimonials' typo, some route naming style differences). Avoid breaking existing paths unless migrating end-to-end.

#14) Known Gaps / Technical Debt

- 'npm run lint' currently reports multiple errors/warnings; project is functional but not lint-clean.
- Some client components use patterns now flagged by stricter React/Next lint rules.
- Route and naming consistency can be improved ('Activities' uppercase route, 'testimonials' spelling).
- Error handling and validation depth varies by endpoint.
- No automated tests are currently present.

#15) Handover Checklist for Incoming Developer

- [] Copy production '.env.local' equivalents securely.
- [] Verify DB connectivity and index/unique constraints (especially 'User.email', 'Blog.slug', 'Service.title').
- [] Verify SMTP settings by testing message/appointment notifications.
- [] Verify Cloudinary credentials and upload/delete behavior.
- [] Smoke-test key admin CRUD flows.
- [] Decide lint policy (fix existing issues vs. relax rule set).
- [] Add API/schema validation and test coverage (recommended next step).

#16) Suggested Immediate Improvements (First Sprint)

1. Add centralized request validation (e.g., zod/yup schemas) for all POST/PUT/PATCH routes.
2. Normalize authorization checks into a shared API middleware/helper.
3. Add integration tests for critical APIs: auth, services, appointments, users.
4. Resolve lint issues and establish CI gate ('lint + test').
5. Standardize route naming (consider alias redirects for backward compatibility).

Prepared for project handover.